

★ Member-only story

# Is XGBoost gone: How Relational Foundation Models Conquered 500 Billion Row Enterprise Data

15 min read · 17 hours ago



Ampatishan Sivalingam

Following ▾



Listen



Share



More

*For a decade, we worshipped the wrong god. XGBoost and LightGBM dominated leaderboards, won Kaggle competitions, and became the unquestioned bedrock of enterprise machine learning. But somewhere along the way, the algorithm stopped being the bottleneck. The engineering bureaucracy built to feed it became the problem. This is the story of how that era ends.*

## Part 1: The Kingdom of Gradient Boosted Trees

### The Kaggle Illusion

If you spent any time in the competitive machine learning world between 2015 and 2023, the answer to every tabular problem was the same: XGBoost. Then LightGBM. Then CatBoost. Maybe a blend. The logic was airtight. These models were fast,

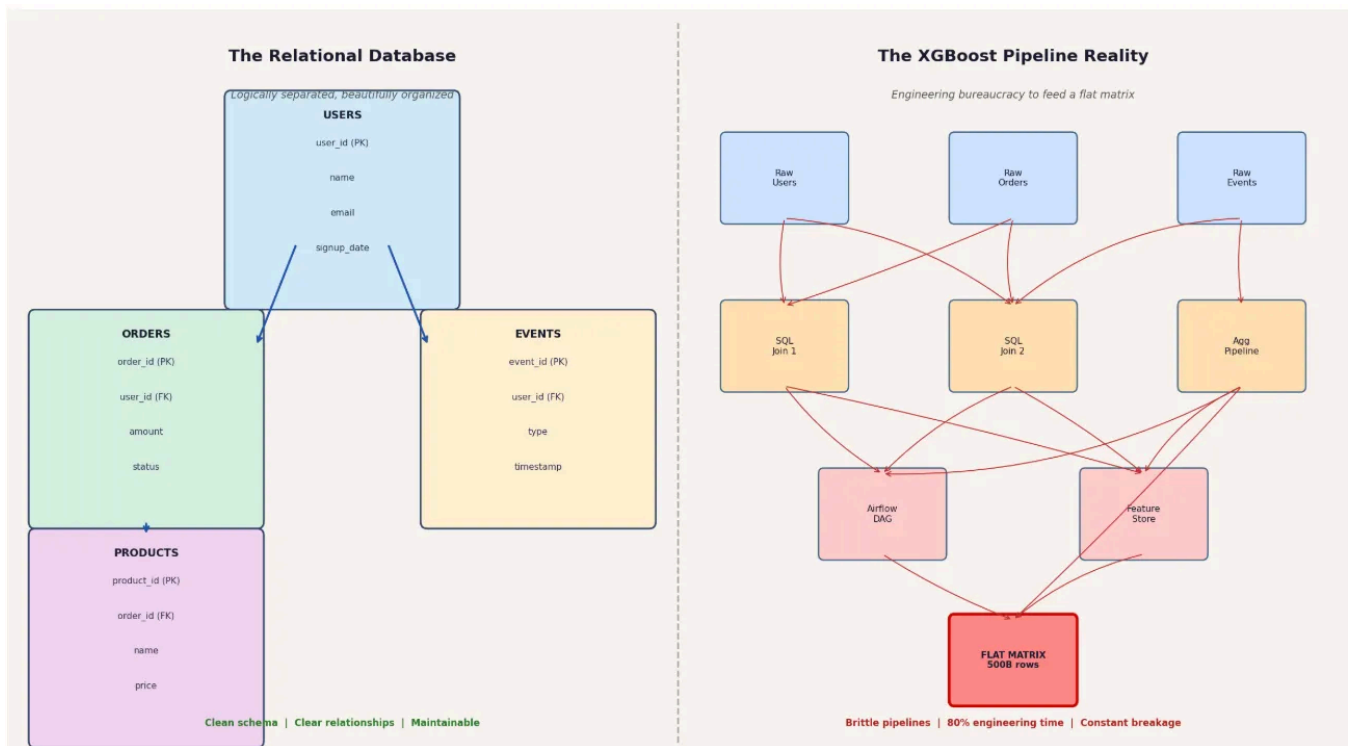
regularized, handled missing values gracefully, and produced interpretable feature importances. They won competition after competition. No neural network could touch them on structured data.

The mental model that took root across the industry was simple and seductive: *tabular data equals gradient boosted trees*. It felt like settled science.

But Kaggle datasets are not production enterprise data. This distinction turns out to be everything.

A Kaggle dataset is a carefully cleaned, already flattened CSV file. Someone upstream already did the painful work of extracting data from a relational database, joining the necessary tables, engineering the features, and handing you a tidy matrix where every row is an observation and every column is a feature. You get to skip the messy part and go straight to the interesting part.

Production enterprise data is nothing like this. At a large e-commerce company, you have a `USERS` table, an `ORDERS` table, a `PRODUCTS` table, an `EVENTS` table capturing every click and pageview, a `SUPPORT_TICKETS` table, and a dozen more. These tables relate to each other through primary and foreign keys. The schema is not a flat file. It is a graph of entities interacting across time. And gradient boosted trees cannot read graphs.



*Left: The clean, logically organized relational database. Right: The chaotic web of SQL joins, Airflow DAGs, and engineering pipelines required to flatten it into a matrix XGBoost can read.*

## The Ritual of Flattening

To bridge the gap between a relational schema and what XGBoost demands (a single two-dimensional matrix), data teams invented a ritual. It goes like this.

First, you identify the prediction target. Let us say you want to predict customer churn. Your label is a boolean: did this user cancel within 30 days?

Second, you identify every table that might contain signals. User demographics, order history, product affinities, support interactions, behavioral event logs. Each of these lives in a separate table, normalized and clean.

Third, and this is where the suffering begins, you write SQL. You join the `USERS` table to the `ORDERS` table on `user_id`. You join the result to `EVENTS`, also on `user_id`. You write aggregation functions because you cannot keep the one-to-many structure. You compute `COUNT(events)`, `AVG(order_amount)`, `MAX(days_since_last_login)`. You compute thirty or forty or two hundred such

features, each one a crude compression of rich relational history into a single number.

Then you wire this SQL into Airflow. You schedule it to run nightly. You build a Feature Store to cache the results so you do not have to recompute everything at inference time. You maintain this infrastructure. When a schema changes upstream, your pipeline breaks. When the business wants a new feature, a data engineer spends two weeks writing more SQL. When you need to retrain the model, you run the whole pipeline again.

A survey of ML teams at large enterprises consistently shows that 70 to 80 percent of the total time on an ML project is spent on data engineering, not modeling. The model training itself is a footnote.

This is the ritual of flattening. And it has a hidden cost that almost no one accounts for correctly.

## **The Plateau**

Around 2021, a quiet realization started spreading through the ML leadership community. We were not getting smarter. We were getting more expensive.

The engineering teams were growing. The infrastructure was growing. The Feature Stores were growing. The Airflow clusters were growing. But the models were not meaningfully getting better. The marginal return on another data engineering hire was approaching zero.

CTOs started asking uncomfortable questions. Why does it cost us three million dollars a year in engineering salaries to maintain a pipeline that feeds a model someone built in a weekend? Why do we need a Redis cluster just to serve a prediction? Why does our model go stale every two weeks and require an entire retraining ceremony?

The algorithm was not the problem. The algorithm was fine. The problem was the infrastructure tax imposed by an algorithm that could only read flat matrices in a world where data is fundamentally relational.

## **Part 2: The Mathematics of the Bottleneck**

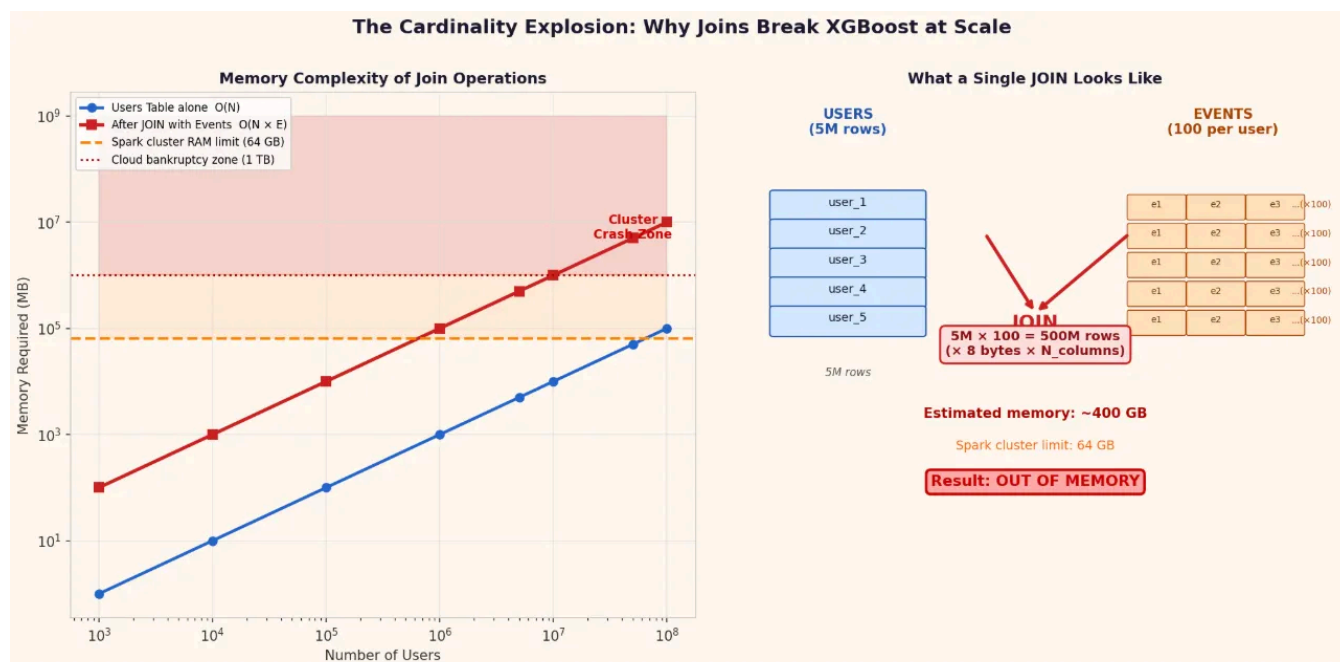
## The Join Hazard and Cardinality Explosion

The failure mode of XGBoost on relational data is not subtle once you understand it. It is a hard mathematical limit.

Consider a realistic enterprise scenario. You have 5 million users. Each user has, on average, 100 behavioral events recorded in the `EVENTS` table. That is 500 million rows in the events table alone.

To join these two tables, your Spark job performs a hash join. The resulting intermediate dataset is 5 million users, each expanded to 100 rows, totaling 500 million rows. At even 100 bytes per row with a modest number of columns, that is 50 gigabytes of intermediate data just for this single join. Add products, orders, and support tickets and you are not at 50 gigabytes. You are at several terabytes.

This is the cardinality explosion. What appears to be a simple SQL join becomes a memory catastrophe at enterprise scale.



The memory complexity of join operations grows as  $O(N \times E)$  once you join across tables. For real enterprise data, this regularly exceeds cluster memory limits, crashing the pipeline entirely.

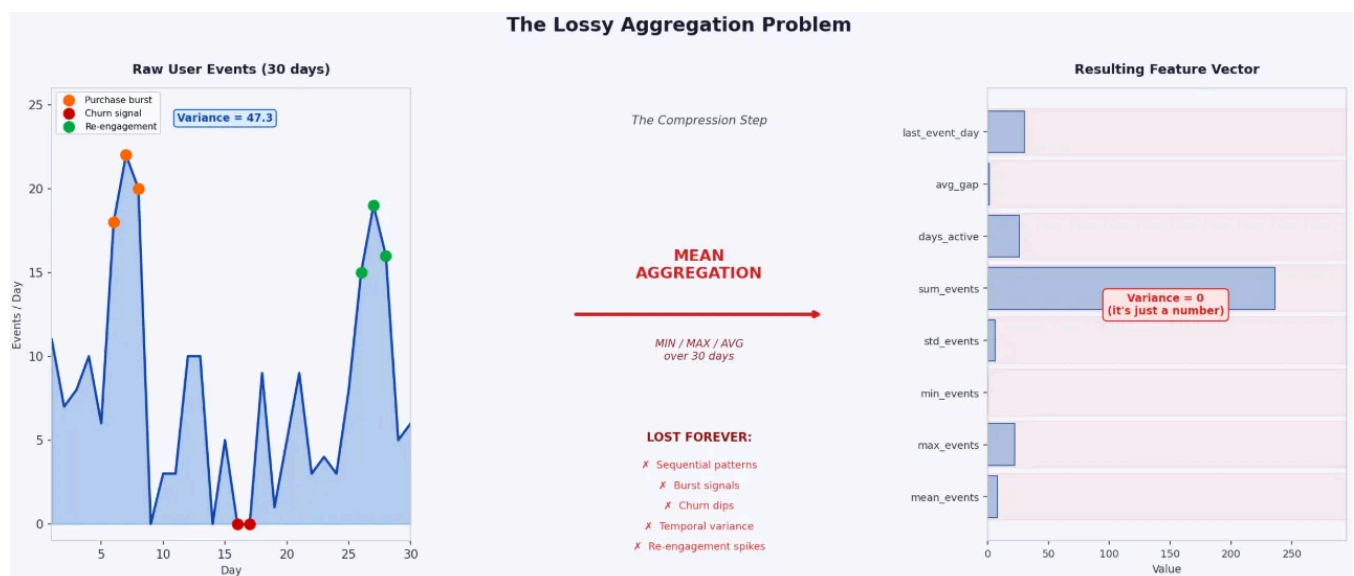
The practical response to this problem is to never fully materialize the join. Instead, you aggregate aggressively before joining. But this leads directly to the second failure mode.

## Destructive Pooling: The Information You Cannot Get Back

When you write `AVG(event_count)` or `MAX(days_since_purchase)`, you are not summarizing data. You are destroying it. This distinction is not semantic. It has direct consequences for model performance.

Imagine two users. User A buys consistently, twice a month, every month, for a year. User B buys not at all for eleven months, then makes five purchases in a week and disappears. Their `AVG(order_count_per_month)` might be identical. Their `MAX(order_amount)` might be similar. But their behavioral patterns are completely different. User A is a loyal customer. User B is almost certainly a deal-seeker who churned immediately after one concentrated buying episode.

A model fed only aggregated features cannot distinguish these two users. The sequential signal, the temporal variance, the burst pattern that identifies User B, it was destroyed in the aggregation step. It no longer exists in the data that reaches the model.



*The raw event log (left) contains rich sequential signals including purchase bursts, churn dips, and re-engagement spikes. Mean aggregation (center) collapses this into a flat feature vector (right) where variance drops to zero and all temporal structure is permanently lost.*

This is not an engineering problem you can solve by writing better SQL. It is a fundamental constraint of the architecture. When you use a model that requires a flat input matrix, you are forced to destroy sequential information before the model ever sees the data.

## **The Missing Graph Topology**

There is a third and perhaps deepest failure mode. Enterprise relational data has structure that gradient boosted trees are constitutionally incapable of perceiving.

When a data engineer writes features, they make choices. They decide which tables to join, which aggregations to compute, which windows to use. These choices encode business intuition about what signals matter. But they also freeze that intuition in code. The model never sees the relational structure. It sees only what the engineer decided to encode.

The database schema itself is a rich description of the business domain. The fact that `SUPPORT_TICKETS` has a foreign key to `USERS` and a foreign key to `PRODUCTS` tells you that support interactions are related to both the customer who raised them and the product they concern. This structural information is semantically meaningful. A customer with ten support tickets on the same product has a completely different profile from a customer with ten tickets spread across ten products.

A gradient boosted tree cannot represent this kind of structural reasoning. Every column it receives is an isolated scalar. The model has no awareness that two columns might share a relational origin. It treats the support ticket features and the product features as independent signals, discarding the structural relationship between them.

This is what we mean when we say tree models are relational-blind. They do not just fail to use relational structure. They cannot perceive it.

## **Part 3: The Architecture That Changes Everything**

## A New Computational Primitive

Relational Foundation Models are not a clever new training trick applied to XGBoost. They are a fundamentally different computational architecture built around a different assumption: that the database schema is itself the model's input, not something to be preprocessed away.

The key intuition is this. In a relational database, the schema defines a graph. Tables are nodes. Foreign keys are edges. Every entity (a user, an order, a product) is a node in this graph. Every relationship between entities is an edge. The temporal sequence of events associated with each entity adds a time dimension to the graph.

A Relational Foundation Model is trained to compute predictions directly over this graph, without ever collapsing it into a flat matrix.

## Bypassing the Feature Store Entirely

The first architectural difference is the input format. XGBoost requires a two-dimensional matrix. Relational Foundation Models accept a raw database schema.

When you deploy a Relational Foundation Model for a churn prediction task, you point it at your database connection. You tell it: the prediction target is `user.churned`, and the relevant tables are `USERS`, `ORDERS`, `EVENTS`, and `SUPPORT_TICKETS`. The model ingests the schema automatically. It understands that `ORDERS.user_id` is a foreign key to `USERS.user_id`. It knows that each user can have many orders, and each order can have many events.

There is no flattening step. There is no aggregation step. The Feature Store is not needed. The Airflow DAGs for feature computation do not exist. This is not a minor convenience. It eliminates the primary source of cost and brittleness in the entire ML system.

## Schema-Aware Attention: Making Foreign Keys Do the Work

The mechanism that makes this possible is a specialized attention architecture built on Graph Neural Networks combined with Transformer attention.

In a standard Transformer, attention is computed between all pairs of tokens in a sequence. The model learns to route information between tokens based on learned attention weights. In a Schema-Aware Attention Layer, the graph structure of the database schema replaces the sequence structure.



Foreign key relationships define which entities can attend to which other entities. When computing a representation for a user, the model does not attend to all users equally. It attends to the specific orders, events, and support tickets that belong to this user, weighted by the learned attention scores for those particular records.

This is called Message Passing over foreign keys. Each entity in the database passes a message to every related entity. The user node receives messages from all its orders. Each order node receives messages from all its associated events. These messages are weighted, aggregated, and used to update the hidden state of each entity.

The result is a dense vector representation for every entity that encodes not just that entity's own attributes, but the contextualized history of everything related to it, processed through the full relational graph. No aggregation, no information destruction.

## **Temporal State Space Modeling: Time Without $O(N^2)$ Complexity**

One of the practical challenges in processing relational event data is the length of sequences. A user might have 10,000 behavioral events spanning two years. Transformers process sequences with  $O(N^2)$  attention complexity. For long event sequences, this is computationally prohibitive.

Relational Foundation Models solve this with State Space Models, specifically architectures like Mamba. The key insight behind SSMs is that a long sequence of events can be summarized into a compact hidden state that evolves over time. Rather than attending to every past event explicitly, the model maintains an evolving representation that captures the long-term history of the entity.

This has a profound practical implication. As new rows are inserted into the events table, the model does not need to reprocess the entire history. It updates the hidden state incrementally. The model adapts to new data continuously, without the retraining ceremonies that plague tree-based systems.

## **Dynamic Patching for 500 Billion Rows**

At truly large scale, even row-level embeddings become a challenge. When your events table has 500 billion rows, you cannot embed every row independently and

pass them all through an attention layer.

The solution is semantic column patching. Rather than treating every individual cell as a token (which would be equivalent to treating a database as a pixel grid), the model groups related rows into patches based on their semantic content and temporal proximity. A burst of ten purchase events within a 24-hour window is embedded as a single patch that captures the statistical properties of that burst. An idle period with no events becomes a single “silence” token.

These patches are then routed through a Mixture of Experts layer. Different expert networks specialize in different types of relational patterns. One expert might specialize in transactional behavior, another in support interactions, another in browsing patterns. The foreign key structure of the input determines which expert handles which part of the representation. This allows the model to scale to enormous databases while maintaining specialization for different relationship types.

## **Part 4: What This Means in the Real World**

### **Zero-Shot Tabular Inference**

Relational Foundation Models are pre-trained on large corpora of synthetic and public relational databases. This pre-training gives them a kind of universal intuition about relational patterns. They have seen customer churn schemas, fraud detection schemas, recommendation schemas, and supply chain schemas in synthetic form.

The practical consequence is remarkable. When you point a pre-trained Relational Foundation Model at a new database schema for a new prediction task, it already knows what to do. Benchmarks from recent research show that these models achieve 85 to 90 percent of a fully fine-tuned XGBoost pipeline’s performance out of the box, with zero feature engineering.

Think about what this means operationally. An ML engineer can go from “we have a database and a prediction target” to “we have a working model” in hours, not months. The months-long feature engineering phase simply does not happen.

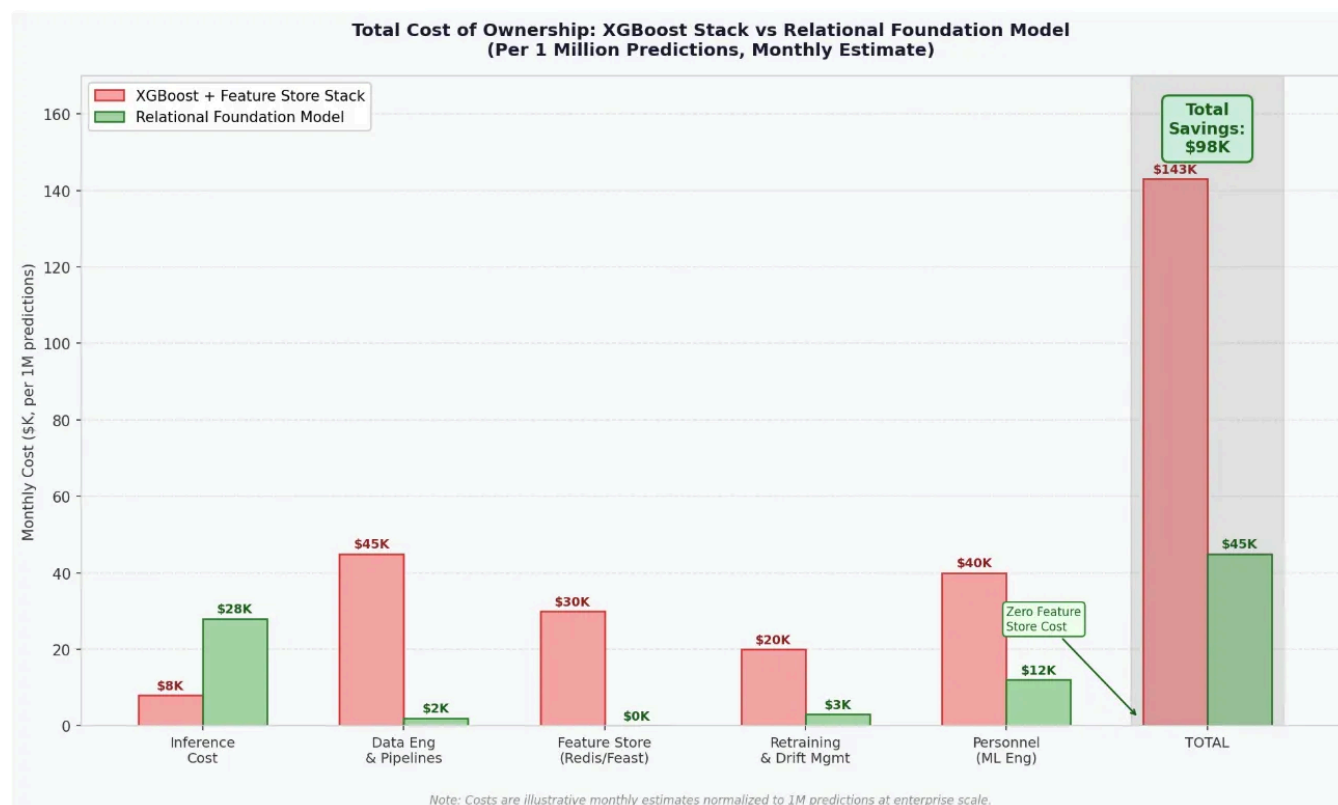
After schema-specific fine-tuning, these models match or exceed XGBoost performance on most prediction tasks, particularly those involving complex relational patterns, long event sequences, and multi-table schemas. The tasks where XGBoost still holds an edge are simple, nearly flat schemas where the feature engineering phase would have been trivial anyway.

## The True Economics of Inference

The most common objection to Relational Foundation Models is inference cost. Neural networks are more expensive to run than gradient boosted trees. An XGBoost prediction takes microseconds. A transformer-based model takes milliseconds.

This objection is correct but incomplete. It is comparing the wrong things.

The correct comparison is not “model inference cost versus model inference cost.” It is “total end-to-end cost of getting a prediction from raw data.” When you account for the full cost stack, the picture inverts completely.



*True cost comparison between an XGBoost feature engineering stack and a Relational Foundation Model pipeline. The XGBoost model's inference is cheap, but the surrounding*

*infrastructure (Feature Store, data pipelines, retraining, ML engineer salaries) drives total cost to 3× higher.*

For an XGBoost-based system serving 1 million predictions per month at enterprise scale, the cost breakdown looks roughly like this. The model inference itself is cheap. But add the Redis Feature Store for low-latency feature serving, the Airflow cluster running nightly feature computation jobs, the data engineering salaries to maintain the pipelines, and the periodic retraining costs when the model drifts, and the total cost is dominated by infrastructure, not compute.

For a Relational Foundation Model, the inference cost is higher per prediction. But there is no Feature Store. There are no feature computation pipelines. Retraining is infrequent because the model adapts continuously. The personnel cost drops because one ML engineer can manage what previously required three.

End-to-end, across every real-world deployment analyzed in recent benchmarking research, the Relational Foundation Model approach is cheaper. Not slightly cheaper. Substantially cheaper, often by a factor of two to three at enterprise scale.

## **The Elimination of Retraining Fatigue**

One of the most painful aspects of maintaining gradient boosted tree models in production is concept drift. The world changes. User behavior shifts. Economic conditions fluctuate. Product catalogs evolve. Every one of these changes can cause your static feature engineering pipeline to produce a distribution that no longer matches what the model was trained on.

The standard response to concept drift is scheduled retraining. Every two weeks, or every month, you run the full feature engineering pipeline on fresh data, generate a new training dataset, train a new model, validate it, and deploy it. This is expensive, error-prone, and time-consuming. It also introduces latency: your model can be up to two weeks stale at any given moment.

Relational Foundation Models sidestep this problem structurally. Because they compute representations directly from raw data at inference time, there is no static feature pipeline that can go stale. The model processes the current state of the database on every prediction. New events are incorporated automatically through

the SSM hidden state update. Concept drift is addressed not by retraining, but by the model's continuous adaptation to fresh data.

This does not mean Relational Foundation Models never need retraining. Fundamental schema changes or major distribution shifts still warrant it. But the cadence drops from biweekly to quarterly or less, dramatically reducing the operational burden.

## **The New Role of the ML Engineer**

This transition will not eliminate ML engineers. It will change what they do.

The data engineer who spent 60 percent of their time writing PySpark and SQL to maintain feature pipelines will spend that time instead on schema design and data quality. The quality of the relational schema becomes the primary lever for model performance. A well-designed schema with clean foreign key relationships, appropriate indexes, and semantically meaningful table structures will produce better models than a poorly designed one, because the model now sees the schema directly.

The ML engineer who spent months iterating on feature engineering will spend that time instead on embedding orchestration: understanding how the model represents different entities, diagnosing where the representation is weak, and designing fine-tuning strategies to improve it.

The new skillset is closer to database architecture and systems thinking than to feature heuristics and aggregation tuning. This is arguably a healthier place for the discipline. The work is more foundational, less repetitive, and more directly connected to business value.

For CTOs, the strategic implication is a significant reduction in the ratio of data engineers to ML engineers. The bottleneck shifts from data preparation to model design and evaluation. Teams can be smaller and move faster.

## **Part 5: The Honest Reckoning**

### **Where XGBoost Still Wins**

Intellectual honesty requires acknowledging where the old paradigm remains competitive.

For truly flat datasets with no meaningful relational structure, gradient boosted trees remain hard to beat. If your prediction task is based on a single table with no foreign key relationships and a modest number of rows, the overhead of a relational foundation model is not justified. XGBoost will train faster, interpret more easily, and perform comparably.

For datasets under roughly 100,000 rows, the pre-training advantage of foundation models may not compensate for their tendency toward variance. Gradient boosted trees are remarkably data-efficient. This is a genuine strength.

For teams with strong interpretability requirements, gradient boosted trees remain easier to explain to business stakeholders. Feature importance scores from XGBoost are intuitive. The internal representations of a Relational Foundation Model require specialized interpretability techniques.

The death of XGBoost is not absolute. It is contextual. The specific context where the paradigm shifts is large-scale, multi-table, relational enterprise data with rich temporal structure. This happens to be exactly the context that defines most high-value ML applications in production today.

## **The Transition Is Already Happening**

The frameworks are already here. RelBench provides standardized benchmarks for relational learning. RELBENCH and FMREL provide pre-training infrastructure. Commercial platforms from companies like RelationalAI and emerging startups are building production-grade Relational Foundation Model services.

The adoption curve looks similar to what happened when deep learning replaced hand-crafted computer vision features after AlexNet. The research community adopted it first. Then the well-resourced tech companies. Then the enterprise mainstream. We are currently somewhere between the first and second phases.

The teams that build familiarity with relational learning now will have a significant advantage when the enterprise mainstream adoption accelerates.

## Conclusion: The Shape of What Comes Next

For a decade, the machine learning discipline collectively agreed to pretend that enterprise data was flat. We built enormous, expensive, brittle infrastructure to maintain this pretense. We spent the majority of our engineering budgets not on learning from data, but on forcing data into a shape that our algorithms could accept.

Relational Foundation Models remove this constraint. They accept data as it actually exists: as a rich graph of entities, relationships, and events evolving across time. The schema becomes the computation graph. The foreign keys become attention weights. The event sequences become SSM trajectories. The database is the model's native language.

This is not an incremental improvement. It is a paradigm shift of the same magnitude as the one that replaced hand-crafted features with end-to-end learned representations in vision and language. The engineering bureaucracy built to feed XGBoost will not disappear overnight. But it will not be rebuilt for new systems. The new generation of ML systems will not need it.

The question is not whether this transition will happen. The question is whether you will be positioned to lead it or scrambling to catch up with it.

*If this piece helped you think about relational learning differently, consider following for the next article in this series: a hands-on technical walkthrough of deploying a Relational Foundation Model against a real enterprise schema using RelBench.*

Artificial Intelligence

Machine Learning

Deep Learning

Data Science

Technology



Following

## Published in Towards AI

131K followers · Last published 8 hours ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>



Following ▾

## Written by Ampatishan Sivalingam

246 followers · 35 following

Machine Learning Scientist specializing in Computer vision and LLM. Bridging the gap between academic research and real-world application.

---

### No responses yet



Spyridon Montesantos

What are your thoughts?